AI Edge (Computing)

3. Quantization in Tensorflow

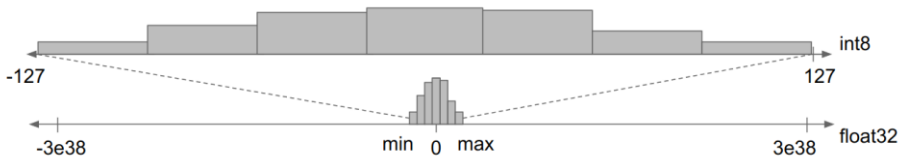
hogeschool vives

1

Quantization

General

- Reducing precision of :
  - weights
  - biases
  - activation functions
- For example to int8, float16



hogeschool vives

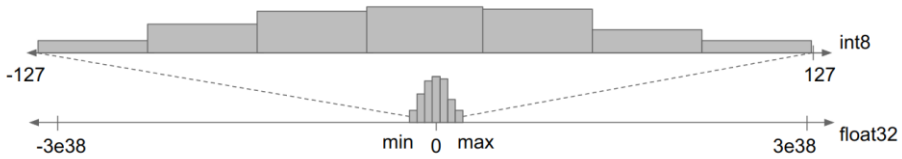
2

Quantization

General

- Goals:
  - Reduce latency, processing, power and model size
  - Minimize model accuracy degradation

$$real_value = (int8_value - zero_point) \times scale$$



3

Quantization

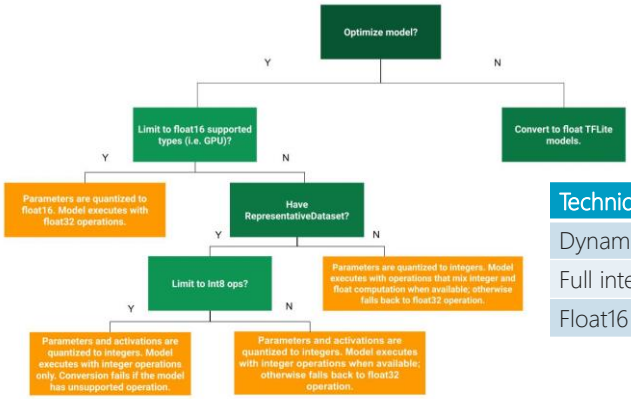
Types

- Post-training Quantization
  - Post-training Dynamic Range Quantization
  - Post-training Full Integer Quantization
  - Post-training Float16 Quantization
- Quantization Aware Training

4

Post-training Quantization (PTQ)

- Can be performed on an already trained model!



Technique	Benefits
Dynamic range quantization	4x smaller, 2x-3x speedup
Full integer quantization	4x smaller, 3x+ speedup
Float16 quantization	2x smaller, possible on GPU

5

Post-training Dynamic Range Quantization

- Good starting point
  - Reduced memory
  - Faster execution/inference
  - No representative/signature dataset necessary
- Although
  - Only quantizes weights & biases from float to int
  - Activation functions are

NETRON

```
import tensorflow as tf
converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_quant_model = converter.convert()
```


6

Post-training Full Integer Quantization

- Full INT8 conversion is required for some microcontrollers and edge TPU's
- Requires a small representative/signature dataset
  - Estimates a dynamic range for the variable data
  - Such as inputs/outputs/activation outputs
  - Determines the scaling from float to int

```
def representative_data_gen():
  for input_value in
tf.data.Dataset.from_tensor_slices(train_images).batch(1).take(100):
    # Model has only one input so each data point has one element.
    yield [input_value]

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_data_gen

tflite_model_quant = converter.convert()
```


7

Post-training Full Integer Quantization

- Problem?!
  - The inputs and outputs are still float32
  - Good for compatibility
  - Bad for edge TPU's
- Some extra lines…
- Todo:
  - Compare sizes!
  - Compare accuracies!

```
def representative_data_gen():
  for input_value in
tf.data.Dataset.from_tensor_slices(train_images).batch(1).take(100):
    yield [input_value]

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_data_gen
# Ensure that if any ops can't be quantized, the converter throws an error
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
# Set the input and output tensors to uint8 (APIs added in r2.3)
converter.inference_input_type = tf.uint8
converter.inference_output_type = tf.uint8

tflite_model_quant = converter.convert()
```


8

Post-training Float16 Quantization

- Reduce size of model
- Increase inference time
- Small loss of precision
- Useful for GPU acceleration if needed

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]
tflite_fp16_model = converter.convert()
```


9

Post-training Quantization → other combinations possible

- 8-bit weights and 16-bit activations
- Useful for:
  - super-resolution,
  - audio signal processing such as noise cancelling and beamforming,
  - image de-noising,
  - HDR reconstruction from a single image
- Still 3x-4x reduction in size
- Compatible with integer only systems

10

Quantization Aware Training (QAT)

- Simulates low-precision inference-time computation in the forward pass of the training process
- Introducing the quantization error as noise during training phase and loss
- The optimalisation algorithm tries to minimize the noise and loss
- Result: model is more robust to quantization

- (almost) the same accuracy as floating point models

- How do we introduce that noise into the model

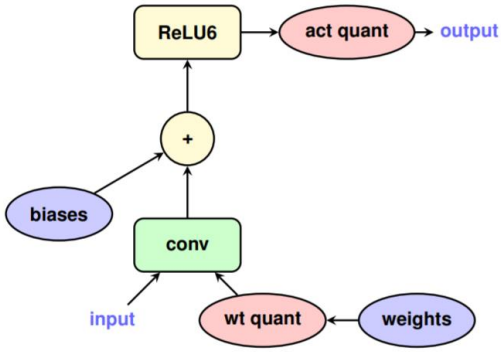
11

Quantization Aware Training (QAT)

- Start from the floating point model
- Add in functions that convert the floating point tensors to integers

- "noise" is introduced
  during forward pass of the model

- Quantize the model partially
- Quantize the model fully



12

Quantization Aware Training (QAT)

- Quantize the model partially

```
import tensorflow_model_optimization as tfmot
quantize_annotate_layer =
tfmot.quantization.keras.quantize_annotate_layer

model = tf.keras.Sequential([
   ...
   # Only annotated layers will be quantized.
   quantize_annotate_layer(Conv2D()),
   quantize_annotate_layer(ReLU()),
   Dense(),
   ...
])

# Quantize the model.
quantized_model = tfmot.quantization.keras.quantize_apply(model)
```


13

Quantization Aware Training (QAT)

- Quantize the model fully

```
import tensorflow_model_optimization as tfmot

model = tf.keras.Sequential([
   ...
])
# Quantize the entire model.
quantized_model = tfmot.quantization.keras.quantize_model(model)

# Continue with training as usual.
quantized_model.compile(...)
quantized_model.fit(...)
```


14

— Now it's your turn!

- Write a script that loads in all the .tflite files and compares their:
  - Size
  - Accuracy
  - Loss
- When running through the validation dataset
- Conclusions?
- Make sure to quantize/dequantize where necessary!

hogeschool vives